

Cyber Toolbox

Manual do Utilizador

February 2, 2026

Contents

1	Visão Geral	3
2	Port Scanner	3
2.1	Uso Básico	3
2.2	Exemplos	3
2.3	Características	3
2.4	Output (exemplo)	3
3	UDP Flooder	4
3.1	Uso Básico	4
3.2	Opções	4
3.3	Exemplos	4
3.4	Aviso Legal	4
4	SYN Flooder	4
4.1	Requisitos	4
4.2	Uso Básico	5
4.3	Opções	5
4.4	Exemplos	5
4.5	O que è um SYN Flood?	5
4.6	Características	5
4.7	Output (exemplo)	6
4.8	Aviso Legal	6
5	Log Analyzer	6
5.1	Uso Básico	6
5.2	Opções	7
5.3	Exemplos	7
5.4	Formatos Suportados	7
5.5	Output (exemplo)	7
5.6	Análise Produzida	7
6	Messenger	8
6.1	Iniciação (Primeira Execução)	8
6.2	Servidor	8
6.3	Cliente - Enviar Mensagem	8
6.4	Cliente - Listar Mensagens	8
6.5	Cliente - Descarregar Mensagem	8

6.6	Cliente - Apagar Mensagem	8
6.7	Cliente - Backup Simétrico	9
6.8	Cliente - Backup Assimétrico	9
6.9	Características Principais	9
7	Port Knocker	9
7.1	Uso Básico	9
7.2	Exemplos	9
7.3	Configuração da Máquina Alvo (Linux)	9
7.4	Fluxo Típico	10
8	Password Manager	10
8.1	Iniciação (Primeira Execução)	10
8.2	Criar Registo	10
8.3	Listar Registos (Requer 2FA)	10
8.4	Visualizar um Registo (Requer 2FA)	10
8.5	Atualizar Registo (Requer 2FA)	11
8.6	Eliminar Registo (Requer 2FA)	11
8.7	Modo Interativo	11
8.8	Características	11
8.9	Ficheiros Criados	11
9	Iniciar via Menu Principal	11
10	Dependências	12
11	Nota de Segurança	12
12	Ficheiros de Log e Relatórios	12

1 Visão Geral

O **Cyber Toolbox** é um conjunto de scripts focados em análise, segurança e administração de sistemas. Este manual descreve o modo de utilização dos principais scripts e as respectivas opções.

2 Port Scanner

Varre portas em uma ou múltiplas máquinas remotas para identificar quais estão abertas.

2.1 Uso Básico

```
python3 port_scanner.py <alvo1,alvo2,...> <porta_inicial> <porta_final>
```

2.2 Exemplos

```
# Escanear um alvo
python3 port_scanner.py 192.168.1.1 1 1024

# Escanear multiplos alvos
python3 port_scanner.py 192.168.1.1,192.168.1.2,google.com 80 443

# Com argumentos nomeados
python3 port_scanner.py --targets 127.0.0.1 --start-port 1 --end-port 65535

# Com timeout customizado
python3 port_scanner.py 127.0.0.1 1 1024 --timeout 5

# Guardar relatorio
python3 port_scanner.py 192.168.1.1 1 1024 --output scan_report.json
```

2.3 Características

- Suporta múltiplos alvos (separados por vírgula)
- Resolução de nomes de hosts
- Identifica serviços associados às portas abertas
- Timeout configurável
- Relatório em JSON (opcional)

2.4 Output (exemplo)

```
60 CYBER-TOOLBOX - PORT SCANNER
=====

Varrendo 192.168.1.1 (192.168.1.1) - Portas 1 a 1024
-----
Porta 22 aberta (ssh)
Porta 80 aberta (http)
Porta 443 aberta (https)

RELATORIO FINAL
=====
```

```
192.168.1.1 (192.168.1.1)
  Portas abertas: 22, 80, 443

Varrimento concluido em 45.32s
```

3 UDP Flooder

Envia pacotes UDP para simular um ataque de negação de serviço (DoS).

3.1 Uso Básico

```
python3 udp_flooder.py --target <ALVO> [OPTIONS]
```

3.2 Opções

```
--target, -t Alvo (IP ou hostname) [obrigatorio]
--port, -p Porta UDP (default: 53)
--duration, -d Duracao em segundos (default: 10)
--packet-size, -s Tamanho do pacote em bytes (default: 1472)
--threads, -n Numero de threads (default: 4)
```

3.3 Exemplos

```
# Flood basico (DNS - porta 53)
python3 udp_flooder.py --target 192.168.1.1

# Flood em porta NTP (123) por 30 segundos
python3 udp_flooder.py --target 192.168.1.1 --port 123 --duration 30

# Flood com 8 threads
python3 udp_flooder.py --target 192.168.1.1 --threads 8

# Flood com pacotes grandes
python3 udp_flooder.py --target 192.168.1.1 --packet-size 65535
```

3.4 Aviso Legal

Este script è **apenas** para fins educacionais em ambientes autorizados (laboratorial). O uso não autorizado è ilegal e pode violar leis de segurança informática.

4 SYN Flooder

Envia pacotes TCP SYN para simular um ataque de negação de serviço (SYN Flood/DoS) contra serviços HTTP, SMTP ou outros.

4.1 Requisitos

```
# Requer Scapy para manipulacao de pacotes raw
pip install scapy

# Requer privilegios de root/administrator
sudo python3 syn_flooder.py --target <ALVO> [OPTIONS]
```

4.2 Uso Básico

```
python3 syn_flooder.py --target <ALVO> [OPTIONS]
```

4.3 Opções

```
--target, -t Alvo (IP ou hostname) [obrigatorio]
--port, -p Porta TCP (default: 80/HTTP)
--duration, -d Duracao em segundos (default: 10)
--threads, -n Numero de threads (default: 4)
```

4.4 Exemplos

```
# SYN Flood basico contra HTTP (porta 80)
sudo python3 syn_flooder.py --target 192.168.1.1

# SYN Flood contra SMTP (porta 25) por 30 segundos
sudo python3 syn_flooder.py --target 192.168.1.1 --port 25 --duration 30

# SYN Flood com 8 threads
sudo python3 syn_flooder.py --target 192.168.1.1 --port 443 --threads 8

# SYN Flood contra servico customizado
sudo python3 syn_flooder.py --target 192.168.1.1 --port 3306 --duration 20
```

4.5 O que è um SYN Flood?

1. Atacante envia muitos pacotes TCP SYN (pedidos de conexão) para um servidor
2. Servidor responde com SYN-ACK, aguardando o ACK final do cliente (three-way handshake)
3. Atacante nunca envia o ACK final (ou falsifica IP de origem)
4. Servidor fica com muitas conexões meio-abertas (*half-open*)
5. Buffer de conexões pendentes enche, servidor nega novas conexões legítimas

4.6 Características

- Suporta múltiplas threads para intensificar o ataque
- IP de origem aleatório (*spoof*) em cada pacote
- Portas de origem aleatórias
- Taxa de pacotes por segundo (pps) no relatório
- Duração configurável
- Resolução de nomes de hosts
- Confirmação de segurança antes de execução

4.7 Output (exemplo)

```
Alvo resolvido: 192.168.1.1 -> 192.168.1.1

=====
AVISO - SYN FLOOD ATTACK (TCP)
=====
Esta prestes a enviar um ataque DoS para: 192.168.1.1:80 (http)
Duracao: 10s | Threads: 4

AVISOS IMPORTANTES:
  1. Este script requer privilegios de root/administrator
  2. Um ataque SYN Flood pode danificar infraestruturas criticas
  3. O uso nao autorizado e ILEGAL em muitas juridicoes
  4. Utilize apenas em ambientes de teste autorizados
-----

Confirma que tem autorizacao? (s/N):

Iniciando SYN Flood (TCP)
  Alvo: 192.168.1.1:80
  Duracao: 10 segundos
  Threads: 4
  Protocolo: TCP SYN (Half-Open Connections)
-----

  500 pacotes SYN enviados para 192.168.1.1:80
  1000 pacotes SYN enviados para 192.168.1.1:80
  1500 pacotes SYN enviados para 192.168.1.1:80

=====
RELATORIO - SYN FLOOD
=====
Alvo: 192.168.1.1:80
Total de pacotes SYN enviados: 5,234
Taxa de envio: 523 pps (pacotes por segundo)
Duracao real: 10.01s
Threads utilizadas: 4
=====
```

4.8 Aviso Legal

Este script è **apenas** para fins educacionais em ambientes autorizados (laboratorial). Um ataque SYN Flood pode danificar infraestruturas críticas e deixar sistemas inacessíveis. O uso não autorizado è **severamente ilegal** e pode resultar em penas de prisão.

5 Log Analyzer

Analisa ficheiros de log (HTTP, SSH, UFW) para extrair informações sobre tentativas de acesso, origem dos acessos e localização geográfica.

5.1 Uso Básico

```
python3 log_analyzer.py <ficheiro1.log> [ficheiro2.log ...] [OPTIONS]
```

5.2 Opções

```
--geoip-db, -g Caminho para GeoLite2-Country.mmdb  
--outdir, -o Diretorio de output (default: reports/)
```

5.3 Exemplos

```
# Analisar ficheiros de log  
python3 log_analyzer.py /var/log/apache2/access.log  
  
# Analisar multiplos ficheiros  
python3 log_analyzer.py /var/log/apache2/access.log /var/log/auth.log  
  
# Com resolucao de pais  
python3 log_analyzer.py /var/log/auth.log --geoip-db /path/to/GeoLite2-City.mmdb  
  
# Especificar diretorio de output  
python3 log_analyzer.py /var/log/auth.log --outdir ./reports
```

5.4 Formatos Suportados

- Apache/Nginx Combined Log: 192.168.1.1 - - [10/Oct/2000:13:55:36 -0700] "GET / HTTP/1.0" 200 1043
- SSH Auth Log: Oct 10 13:55:36 host sshd[123]: Failed password for invalid user admin from 192.168.1.1 port 53840 ssh2
- UFW Firewall Log: Oct 10 13:55:36 host UFW BLOCK IN=eth0 OUT= SRC=192.168.1.1 DST=...

5.5 Output (exemplo)

```
Log Analyzer - Analise de Registos  
  
Ficheiro processado: /var/log/auth.log  
156 eventos extraidos  
  
Summary:  
Services:  
  ssh: 156  
Top countries:  
  China: 45  
  Russia: 32  
  United States: 28  
  ...  
  
Relatorios guardados:  
- auth_report.json  
- auth_report.csv
```

5.6 Análise Produzida

- Localização por país (se GeoLite2 disponível)
- Origem dos acessos (IPs)
- Timestamps de tentativas de acesso

- Status de autenticação (sucesso/falha)
- Relatórios em JSON e CSV

6 Messenger

Serviço seguro de troca de mensagens entre cliente e servidor usando criptografia simétrica e assimétrica.

6.1 Iniciação (Primeira Execução)

```
# Gerar chaves RSA
python3 messenger.py init-keys
```

6.2 Servidor

```
# Iniciar servidor na porta padrao (9009)
python3 messenger.py serve

# Com host e porta customizada
python3 messenger.py serve --host 0.0.0.0 --port 5555
```

6.3 Cliente - Enviar Mensagem

```
python3 messenger.py send \
    --from alice \
    --to bob \
    --message "Ola Bob!" \
    --host 127.0.0.1 \
    --port 9009
```

6.4 Cliente - Listar Mensagens

```
python3 messenger.py list --user alice --host 127.0.0.1 --port 9009
```

6.5 Cliente - Descarregar Mensagem

```
python3 messenger.py download \
    --user alice \
    --id <message-id> \
    --host 127.0.0.1 \
    --port 9009
```

6.6 Cliente - Apagar Mensagem

```
python3 messenger.py delete \
    --user alice \
    --id <message-id> \
    --host 127.0.0.1 \
    --port 9009
```


6.7 Cliente - Backup Simétrico

```
python3 messenger.py backup-sym \  
    --password minha_senha_secreta \  
    --host 127.0.0.1 \  
    --port 9009
```

6.8 Cliente - Backup Assimétrico

```
python3 messenger.py backup-async --host 127.0.0.1 --port 9009
```

6.9 Características Principais

- Mensagens encriptadas com criptografia híbrida (Fernet + RSA)
- Suporte a múltiplos utilizadores
- Arquivo de mensagens no servidor
- Backup com encriptação simétrica (com senha) ou assimétrica (com chave pública)
- Descarga seletiva de mensagens
- Eliminação de mensagens apenas por participantes

7 Port Knocker

Cliente para *port knocking* - técnica de abertura de porta SSH através de uma sequência de pacotes UDP.

7.1 Uso Básico

```
python3 port_knocker.py <host> <porta1> [porta2] [porta3] ...
```

7.2 Exemplos

```
# Sequencia de knocking simples  
python3 port_knocker.py 192.168.1.100 7000 8000 9000  
  
# Com hostname  
python3 port_knocker.py exemplo.com 1000 2000 3000 4000  
  
# Uma unica porta  
python3 port_knocker.py 192.168.1.100 5555
```

7.3 Configuração da Máquina Alvo (Linux)

Instale um serviço de port knocking no servidor. Exemplo com **knockd**:

```
# Instalar knockd  
sudo apt-get install knockd  
  
# Configurar /etc/knockd.conf  
[options]  
    logfile = /var/log/knockd.log  
  
[openSSH]
```

```
sequence = 7000,8000,9000
seq_timeout = 15
command = /sbin/iptables -I INPUT -s %IP% -p tcp --dport 22 -j ACCEPT
tcpflags = syn
```

[closeSSH]

```
sequence = 9000,8000,7000
seq_timeout = 15
command = /sbin/iptables -D INPUT -s %IP% -p tcp --dport 22 -j ACCEPT
tcpflags = syn
```

7.4 Fluxo Típico

1. Servidor tem SSH bloqueado na firewall
2. Cliente executa: `python3 port_knocker.py servidor.com 7000 8000 9000`
3. Servidor detecta a sequência de pacotes
4. Servidor abre porta 22 (SSH) para o IP do cliente
5. Cliente pode agora fazer SSH normalmente

8 Password Manager

Gestor de passwords simples com encriptação RSA + Fernet e autenticação de 2 fatores (TOTP).

8.1 Iniciação (Primeira Execução)

```
python3 password_manager.py --init
```

Isto irá:

- Gerar par de chaves RSA
- Gerar segredo TOTP e URI de provisioning
- Criar base de dados vazia

Importante: Adicione o segredo TOTP à sua app autenticadora (Google Authenticator, Authy, Microsoft Authenticator, etc.).

8.2 Criar Registo

```
python3 password_manager.py \  
--create \  
--url example.com \  
--user admin \  
--pass senha123
```

8.3 Listar Registos (Requer 2FA)

```
python3 password_manager.py --list --otp 123456
```

8.4 Visualizar um Registo (Requer 2FA)

```
python3 password_manager.py --view <id> --otp 123456
```

8.5 Atualizar Registo (Requer 2FA)

```
python3 password_manager.py \  
    --update <id> \  
    --url novositeexemplo.com \  
    --user novo_user \  
    --pass nova_senha \  
    --otp 123456
```

8.6 Eliminar Registo (Requer 2FA)

```
python3 password_manager.py --delete <id> --otp 123456
```

8.7 Modo Interativo

```
python3 password_manager.py
```

O menu interativo apresenta as opções para todas as operações.

8.8 Características

- Criptografia assimétrica (RSA) para as chaves
- Criptografia simétrica (Fernet) para os dados
- Autenticação de 2 fatores (TOTP) obrigatória para acesso
- Operações CRUD completas
- Armazenamento seguro de 3 campos: URL, utilizador, password
- Modo interativo ou linha de comando

8.9 Ficheiros Criados

```
private_key.pem (Chave privada - MANTEM SECRETO!)  
public_key.pem (Chave publica)  
totp_secret.txt (Segredo TOTP - guarde)  
records.json (Base de dados encriptada de registos)
```

9 Iniciar via Menu Principal

Todos os scripts podem ser executados através do menu principal:

```
python3 menu_launcher.py
```

O menu apresenta:

```
===== MENU PRINCIPAL - CYBER TOOLBOX =====  
1 - Port Scanner  
2 - UDP Flooder  
3 - SYN Flooder  
4 - Log Analyzer  
5 - Messenger  
6 - Port Knocker  
7 - Password Manager  
0 - Sair  
=====
```

10 Dependências

Instale as dependências com:

```
pip install -r requirements.txt
```

Ou manualmente:

```
pip install geoip2 requests cryptography pyotp
```

11 Nota de Segurança

Alguns scripts requerem privilégios elevados (root/administrator) devido ao acesso a sockets brutos:

```
sudo python3 udp_flooder.py --target 192.168.1.1  
sudo python3 menu_launcher.py
```

12 Ficheiros de Log e Relatórios

- Logs: `src/logs/`
- Relatórios: `reports/`
- Dados do Password Manager: `src/scripts/`
- Dados do Messenger: `src/data/messages/`, `src/data/keys/`, `src/data/backups/`